Page No.
Date

Subroutine is a sequence of program instructions that perform a specific task, packaged as a unit.
It is implemented by using CALL & RETURN instructions.

Branching instructions in 8085

The three types of branching instructions in 8085 are:

- Jump [unconditional and conditional]
- Call [unconditional and conditional]
- Return [unconditional and conditional]

Call statements	Return statements	Call if / Return if
CALL address	RET	Unconditional
CC address	RC	if carry
CNC address	RNC	if no carry
CZ address	RZ	if zero
CNZ address	RNZ	if no zero
CP address	RP	on plus sign
CM address	RM	on minus sign
CPO address	RPO	on parity odd
CPE address	RPE	on parity even

Page No.
Date

Stack pointer

It is a special purpose register of 16 bit width, which holds the address of the top of the stack.

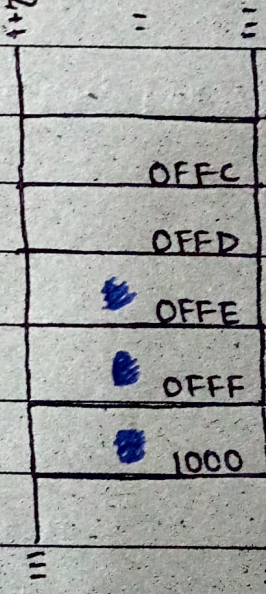
The stack is then implemented by using PUSH & POP instructions.

- LXI SP XXXXH (16 bit addr) // syntax to initialise stack pointer

- PUSH rp // SP-- & content of reg pair → top of stack
- POP rp // content of reg pair ← top of stack & SP++

Q To swap the content of BC & DE reg pairs using stack.

LXI SP, 1000H	SP = 1000
PUSH B	SP = OFFF, M[OFFF] = B
	SP = OFFE, M[OFFE] = C
PUSH D	SP = OFFD, M[OFFD] = D
	SP = OFFC, M[OFFC] = E
POP B	C = M[OFFC], SP = OFFD
	B = M[OFFD], SP = OFFE
POP D	E = M[OFFE], SP = OFFF
	D = M[OFFF], SP = 1000



You can use XCHG instruction to exchange the content of DE & HL reg pairs.

Page No.
Date

Q To perform multiplication using subroutine

```
LHLD  2050H
MOV   A,H
MOV   E,L
CALL  MULT
SHLD  2052H
HLT
```

Use only one of the given subroutines

(by shifting multiplicand)

```
MULT: MVI  D,00H
      MVI  C,08H
      LXI  H,0000H
loop: RAR
      JNC  skip
      DAD  D
skip: XCHG
      DAD H
      XCHG
      DCR C
      JNZ loop
      RET
```

(by shifting product)

```
MULT: MVI  D,00H
      MVI  C,08H
      LXI  H,0000H
loop: DAD  H
      RAL
      JNC  skip
      DAD  D
skip: DCR  C
      JNZ  loop
      RET
```

Page No.
Date

Q To find fibonacci series upto 10 terms

```
        LXI   H, 2050 H
        MVI   A, 00 H
        MOV   M, A
        INX   H
        MVI   B, 01H
        MOV   M, B
        MVI   C, 08 H
back:   INX   H
        ADD   B
        MOV   M, A
        MOV   A, B
        MOV   B, M
        DCR   C
        JNZ   back
        HLT
```

2050	00
2051	01
2052	01
2053	02
2054	03
2055	05
2056	08
2057	0D
2058	15
2059	22
205A	~~37~~

Q To find the factorial of a number (fact(n) ≤ 16 bits)

```
        LDA   2050H
        MOV   B,A
        LXI   D,0001H
again:  CALL  FACT
        DCR   B
        JNZ   again
        XCHG
        SHLD  2051H
        HLT

FACT:   MOV   A,B
        MVI   C,08H
        LXI   H,0000H
back:   DAD   H
        RAL
        JNC   next
        DAD   D
next:   DCR   C
        JNZ   back
        RET
```

Page No.
Date

AF & PSW

AF is a register pair formed with Accumulator & Flag registers and also known as PSW (Processor - Status Word). In PSW, Accumulator is the MS Byte, and Flag register is the LS Byte.

Accumulator	S	Z	x	AC	x	P	x	Cs

AF / PSW

Q To set all flags 1.

```
LXI  SP, 0000H
MOV  B, A
MVI  C, FFH
PUSH B
POP  PSW
HLT
```

FFFE	F F
FFFF	content of A

A ← B ← A

PSW ← C ← FF

To set all flags zero, take MVI C, 00 H in line 3.

- STC // Set Carry Flag
- CMC // Compliment Carry Flag
- CMA // Compliment Accumulator

1s compliment & 2s compliment of a Binary Number

- 1s compliment of a binary number is another binary number obtained by toggling all bits in it.
- 2s compliment of a binary number is 1 added to its 1s compliment

eg: number = 1 0 1 0 0 1 0 0
1s compliment = 0 1 0 1 1 0 1 1
2s compliment = 0 1 0 1 1 1 0 0

Q To find 1s and 2s compliment of a number

```
LXI   H,2050H
MOV   A,M
CMA
INX   H
MOV   M,A
ADI   01H
INX   H
MOV   M,A
HLT
```

2050	number	→
2051	1s comp	←
2052	2s comp	←

Page No.
Date

- ORA r/M // OR the content of Reg with accumulator & store the result back in accumulator.
- ANA r/M // AND the content of Reg with accumulator & store the result back in accumulator.

eg: Let A 1010 0100
B 1001 1001
ORA B 1011 1101
ANA B 1000 0000
XRA B 0011 1101

- XRA r/M // XOR the content of Reg with accumulator & store the result back in accumulator
- ORI XX H // OR the 8 bit data with A & store the result back in A
- ANI XX H // AND the 8 bit data with A & store the result back in A
- XRI XX H // XOR the 8bit data with A & store the result back in A

XRI FFH ≡ CMA ≡ 1s complument of A

ORI 00H ≡ ANI FFH ≡ no change

Page No.
Date

Input and Output ports

IN and OUT instructions are for communication with I/O ports. Both instruction mnemonics are followed by an 8bit port addr, thus we can have (2^8) 256 input ports & 256 output ports.

- IN 8bit Port # // Accumulator ⟵ PORT
- OUT 8bit Port # // Accumulator ⟶ PORT

Delay and Counter

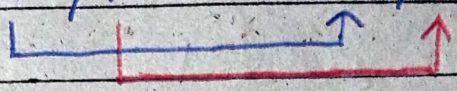
Executing some instructions in loop, the instructions execution times add up to generate a delay.

Read from memory instructions take 3T cycles
Write on memory instructions take 3T cycles
instruction execution take 1T cycles

eg: MVI B,08H ⟶ 2000:06 2001:08

7T ⟵ 3T + 3T + 1T

MVI	C, FFH	7T * 1 time
back: DCR	C	4T * 255 time
JNZ	back	10T/7T * 254/1 time



delay = [(7*1) + (4*255) + (10*254) + (7*1)] T
= [(7*1) + (14*255) - 3] T
= 3574 T

here, T = 1/frequency
JNZ back takes 10T cycles when it jumps back & 7T cycles in the last iteration

Q To generate a delay of 1 ms, given the frequency = 2MHz

some 8 bit value

MVI C, count	MVI C, 8FH
back: DCR C	back: DCR C
JNZ back	JNZ back

delay = [7 + (14*count) - 3] × 1/2MHz = 1ms
= [14 count + 4] × 0.5 × 10^{-6} s = 1ms
= (7 count + 2) μs = 1000μs

⇒ 7 count + 2 = 1000
count ≈ $(143)_{10}$ = 8F ⇒ put count = 8F

* icount is an 8bit number
* jcount is a 16bit number.

Page No.
Date

Q To generate a delay of 1 sec, take f = 2MHz

```
        MVI  D, icount*H     7T
iback:  LXI  B, jcount#H     10T
jback:  DCX  B               6T
        MOV  A, C            4T
        ORA  B               4T
        JNZ  jback           10T/7T
        DCR  D               4T
        JNZ  iback           10T/7T
```

jloop: DCX B, MOV A,C, ORA B, JNZ jback

iloop: LXI B, jcount#H, jloop, DCR D, JNZ iback

$T_j = (6+4+4+10)*(jcount-1) + (6+4+4+7)\ T$
$= (6+4+4+10)*jcount - 3\ T$
$= 24*jcount - 3\ T$

$T_i = (10+T_j+4+10)*(icount-1) + (10+T_j+4+7)$
$= (10+T_j+4+10)*icount - 3\ T$
$T_i = (24+T_j)*icount - 3\ T$

$Delay = 7 + T_i = 7 + (24+T_j)*icount - 3$
$= (24+T_j)*icount + 4$

Let icount = 02H
$\Rightarrow Delay = (24+T_j)*2 + 4 = 2*T_j + 52$

1010 0010, 1100 0011,
A 2 C 3

Page No.
Date

Delay = $2*T_j + 52 = 2*(24*jcount) - 3$
$= (48*jcount - 3) * 1/2MHz$

putting, $(48*jcount - 3) * 0.5 * 10^{-6} s = 1s$
$24*jcount - 1.5 = 10^6$
$24 jcount \approx 10^6$
$jcount = 10^6/24 = 41666.66$
$jcount = 41667$
$jcount = A2C3\ H$

putting icount = 02H & jcount = A2C3H

```
      MVI   D, 02H
iback: LXI   B, A2C3H
jback: DCX   B
       MOV   A, C
       ORA   B
       JNZ   jback
       DCR   D
       JNZ   iback
```

$T_j = (24*41667 - 3)T = 1000005\ T$
$T_i = (24 + T_j)*icount - 3)T = 2000055T$
$delay = 7 + T_i = 2000062\ T$
≈ 1.000031 sec

Q To count continuously in hexadecimal from FFH to 00H, in a system with 0.5 μs clock period. Use C register to setup 1 ms delay between each count and display the count at one of the output port.

```
        MVI   B,00H        7T
next:   DCR   B            4T
        MVI   C, counter   4T
delay:  DCR   C            4T
        JNZ   delay        10T/7T
        MOV   A,B          4T
        OUT   port#        10T
        JMP   next         10T
```

i loop

j loop

delay between two outputs

$= (10 + 4 + 4 + T_j + 4 + 10)\ T = T_i$

$T_j = (4+10)^* (counter - 1) + (4+7)$

$= (4+10)^* counter - 3$

T_i

$delay = (32 + (4+10)^* counter - 3)\ T$

$= (29 + 14^* counter) * 0.5 \times 10^{-3}$ ms $= 1$ ms

$= 15 + 7^* counter = 10^3$

Page No.
Date

$\Rightarrow$ 7* counter = $10^3 - 15$ = 985

counter = 985/7 = 141

counter = 8D H

putting counter = 8D H

```
        MVI   B,0DH
next:   DCR   B
        MVI   C,8DH
delay:  DCR   C
        JNZ   delay
        MOV   A,B
        OUT   port#
        JMP   next
```

delay = $(29 + 14 * 141) * 0.5 * 10^{-3}$ ms

= $1001.5 * 10^{-3}$ ms

delay = 1.0015 ms